МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Кафедра оптоелектроніки та інформаційних технологій

Звіт

До лабораторної роботи №1

Варіант 4

На тему "Основи функціонального програмування у Python"

Виконала:

Студентка групи ФЕП-21

Завалій А. Є.

Перевірив:

Ас. Остріков Д. А.

Львів 2026

Мета роботи:

1. зрозуміти основні принципи функціонального програмування, зокрема чисті функції та референтну прозорість;
2. навчитися виявляти й ізолювати побічні ефекти;
3. реалізувати обробку кошика покупця у функціональному стилі без зміни вхідних даних;
4. застосувати typing.Callable для параметризації обчислень і створення функцій вищого порядку;
5. закріпити навички тестування чистих функцій за допомогою pytest.

Теоретичні відомості:

- **Чиста функція:** функція, яка для однакових вхідних даних завжди повертає однаковий результат, не змінює вхідні дані чи глобальний стан і не виконує операцій введення-виведення.
- **Референтна прозорість:** властивість виразу, за якої його можна замінити отриманим значенням без зміни поведінки програми.
- **Побічні ефекти:** дії, які впливають на зовнішнє середовище або стан програми, наприклад виведення даних через `print()`, робота з файлами, зміна вхідних структур або використання глобальних змінних.
- **Функції вищого порядку:** функції, які приймають інші функції як аргументи або повертають їх як результат.
- **`typing.Callable`**: засіб типізації Python, який дозволяє визначати типи функцій, їхні аргументи та значення, що вони повертають.
- **Функціональний підхід:** передбачає відокремлення чистої логіки обчислень від побічних ефектів. У цій роботі розрахунок вартості товарів у кошику виконується окремо від виведення результатів на екран.

Хід роботи:

Під час виконання лабораторної роботи було розроблено програму для розрахунку вартості товарів у кошику покупця. Для кожного товару зберігаються назва, ціна, кількість і категорія.

У модулі `core.py` реалізовано функції для обчислення вартості окремого товару та загальної вартості кошика. Також створено функцію `make_processor`, яка приймає три функції як аргументи: для застосування знижки залежно від категорії товару, використання купона та нарахування податку. Для їх типізації використано `typing.Callable`.

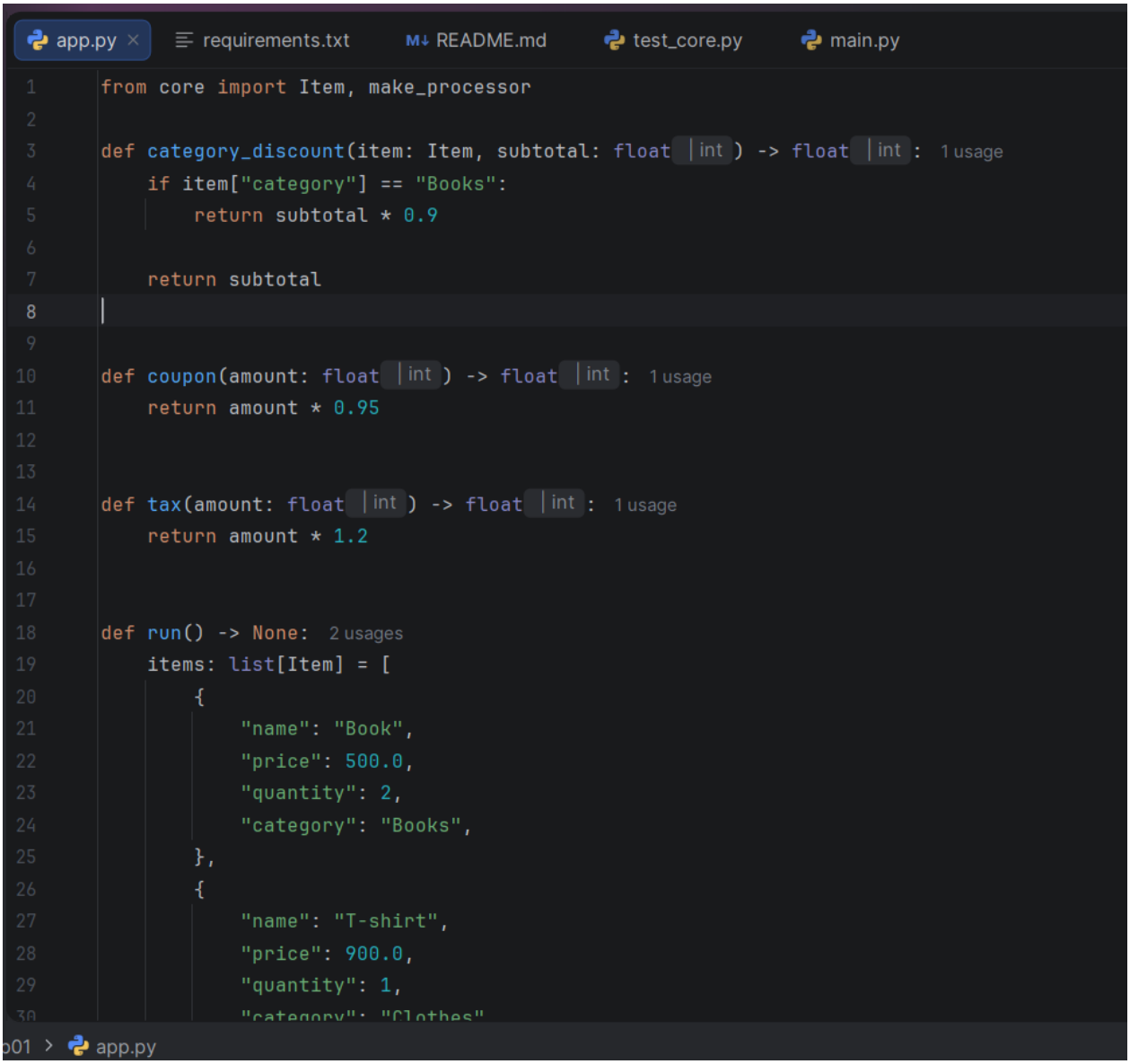
Під час обробки кошика для кожного товару послідовно обчислюється початкова вартість, застосовується категорійна знижка, знижка за купоном і податок. Результати зберігаються в нових словниках із додатковим полем `total`, тому початкові дані не змінюються. Після обробки формується

результат, який містить список товарів, початкову вартість кошика та загальну суму з урахуванням усіх нарахувань.

У модулі `app.py` визначено конкретні правила обчислень: для товарів категорії Books застосовується знижка 10 %, купон зменшує вартість на 5 %, а податок збільшує суму на 20 %. Також створено приклад кошика з товарами різних категорій для демонстрації роботи програми.

Для перевірки правильності роботи програми створено автоматизовані тести за допомогою `pytest`. Вони перевіряють обчислення вартості окремого товару та всього кошика, незмінність вхідних даних, референтну прозорість, правильність застосування переданих функцій обробки та коректну роботу програми з порожнім кошиком.

1. Файл app.py



```
        {
            "name": "T-shirt",
            "price": 900.0,
            "quantity": 1,
            "category": "Clothes",
        },
    ]

    process_cart = make_processor(
        category_discount,
        coupon,
        tax,
    )

    result = process_cart(items)

    print("Shopping cart:")

    for item in result["items"]:
        print(item)

    print("Subtotal:", result["subtotal"])
    print("Total with VAT:", round(result["total"], 2))


if __name__ == "__main__":
    run()
```

2. Файл core.py

```
from typing import Callable, TypedDict

class Item(TypedDict):  14 usages
    name: str
    price: float | int
    quantity: int
    category: str

class ItemWithTotal(Item):  3 usages
    total: float | int

class CartResult(TypedDict):  2 usages
    items: list[ItemWithTotal]
    subtotal: float | int
    total: float | int

DiscountFn = Callable[[Item, float], float]
CouponFn = Callable[[float], float]
TaxFn = Callable[[float], float]

def item_subtotal(item: Item) -> float | int:  5 usages
    return item["price"] * item["quantity"]

def cart_subtotal(items: list[Item]) -> float | int:  2 usages
    return sum(item_subtotal(item) for item in items)

def with_total(item: Item, total: float | int) -> ItemWithTotal:  1 usage
    return {**item, "total": total}
```

```python
def make_processor(  4 usages
    category_discount: DiscountFn,
    coupon: CouponFn,
    tax: TaxFn,
) -> Callable[[list[Item]], CartResult]:

    def process(items: list[Item]) -> CartResult:
        new_items: list[ItemWithTotal] = []

        for item in items:
            subtotal = item_subtotal(item)
            discounted = category_discount(item, subtotal)
            after_coupon = coupon(discounted)
            total = tax(after_coupon)

            new_items.append(with_total(item, total))

        return {
            "items": new_items,
            "subtotal": cart_subtotal(items),
            "total": sum(item["total"] for item in new_items),
        }

    return process
```

3. Файл test_core.py

```python
from copy import deepcopy

from core import (
    Item,
    item_subtotal,
    cart_subtotal,
    make_processor,
)

def test_item_subtotal() -> None:
    item: Item = {
        "name": "Book",
        "price": 500.0,
        "quantity": 2,
        "category": "Books",
    }

    assert item_subtotal(item) == 1000.0

def test_cart_subtotal() -> None:
    items: list[Item] = [
        {
            "name": "Book",
            "price": 500.0,
            "quantity": 2,
            "category": "Books",
        },
        {
            "name": "T-shirt",
            "price": 900.0,
```

```
        {
            "name": "T-shirt",
            "price": 900.0,
            "quantity": 1,
            "category": "Clothes",
        },
    ]
    assert cart_subtotal(items) == 1900.0

def test_input_is_not_modified() -> None:
    items: list[Item] = [
        {
            "name": "Book",
            "price": 500.0,
            "quantity": 2,
            "category": "Books",
        }
    ]
    original = deepcopy(items)

    processor = make_processor(
        lambda item : Item , amount : float | int : amount * 0.9,
        lambda amount : float | int : amount * 0.95,
        lambda amount : float | int : amount * 1.2,
    )
```

```
    processor(items)
    assert items == original

def test_referential_transparency() -> None:
    item: Item = {
        "name": "Book",
        "price": 500.0,
        "quantity": 2,
        "category": "Books",
    }

    assert item_subtotal(item) == item_subtotal(item)

def test_processor_uses_policies() -> None:
    items: list[Item] = [
        {
            "name": "Book",
            "price": 400.0,
            "quantity": 1,
            "category": "Books",
        }
    ]
```

```python
    processor = make_processor(
        lambda item : Item , amount : float | int : amount - 10,
        lambda amount : float | int : amount - 5,
        lambda amount : float | int : amount * 1.2,
    )
    result = processor(items)
    assert result["items"][0]["total"] == 462.0

def test_empty_cart() -> None:
    processor = make_processor(
        lambda item : Item , amount : float | int : amount,
        lambda amount : float | int : amount,
        lambda amount : float | int : amount * 1.2,
    )
    result = processor([])
    assert result["items"] == []
    assert result["subtotal"] == 0
    assert result["total"] == 0
```

4. main.py

```python
"""Точка входу лабораторної роботи №1, варіант 4."""

from app import run


def main() -> None:  1 usage
    """Запуск програми розрахунку кошика покупця."""
    run()

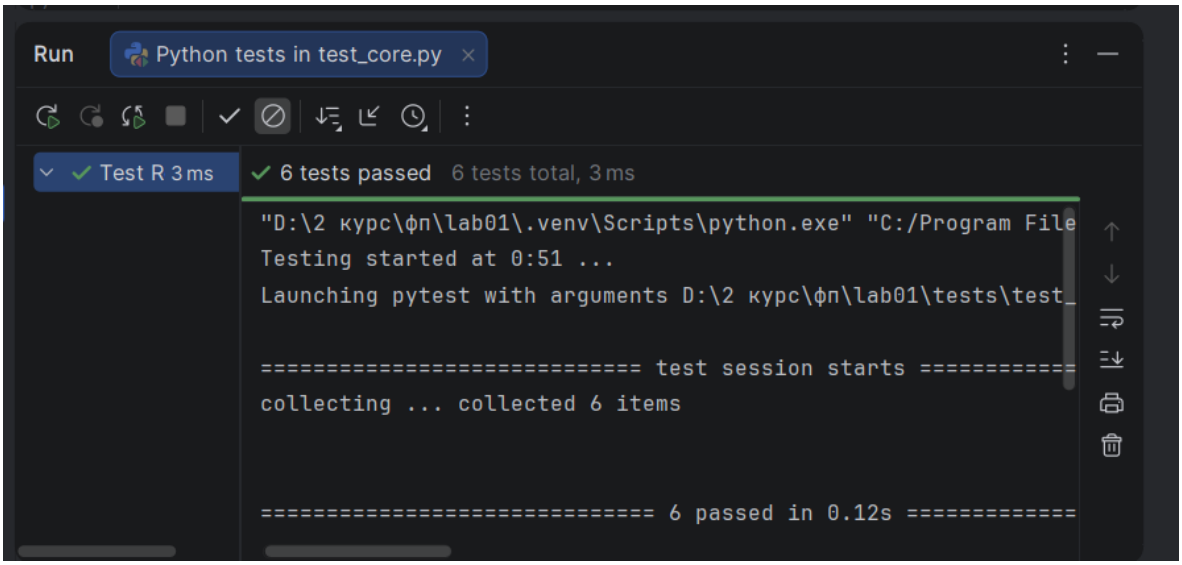

if __name__ == "__main__":
    main()
```

5. Запуск програми

```
Run  app
"D:\2 курс\фп\lab01\.venv\Scripts\python.exe" "D:\2 курс\фп\lab01\app.py"
Shopping cart:
{'name': 'Book', 'price': 500.0, 'quantity': 2, 'category': 'Books', 'total': 1(
{'name': 'T-shirt', 'price': 900.0, 'quantity': 1, 'category': 'Clothes', 'total
Subtotal: 1900.0
Total with VAT: 2052.0

Process finished with exit code 0
```

6. Запуск тестів



Висновок: Під час виконання лабораторної роботи було розроблено програму для розрахунку вартості товарів у кошику покупця з використанням принципів функціонального програмування в Python. Обчислювальну логіку відокремлено від введення та виведення даних, а основні операції реалізовано у вигляді функцій, результати яких залежать від переданих аргументів.

За допомогою typing.Callable реалізовано передавання правил застосування категорійних знижок, купонів і нарахування податку як функцій-аргументів. Програма формує нові записи з підсумковою вартістю товарів, не змінюючи початковий список. Автоматизовані тести за допомогою pytest дають змогу перевірити правильність обчислень, незмінність вхідних даних і стабільність результатів.

Отже, під час виконання лабораторної роботи було закріплено навички використання чистих функцій, функцій вищого порядку та принципів функціонального програмування.